

TASKTRACKER

Documentação Técnica do Projeto

Da concepção lógica à conteinerização com Docker

Aluno: Eduardo Henrique Moraes Santos

RA: 22309110

Disciplina: Bootcamp II

Ciência de Dados e Machine Learning - Campus da Asa Norte

Projeto cumulativo — Fases 1, 2 e 3

Sumário

1. Como ler este documento
2. Visão geral do sistema
3. Fase 1 — O problema e a solução proposta
4. Fase 1 — Decomposição dos dados e regras de negócio
5. Fase 1 — O algoritmo do sistema
6. Fase 1 — Mapeamento arquitetural do repositório
7. Fase 2 — Implementação em Python
8. Fase 2 — Versionamento com Git e GitHub
9. Fase 3 — Conteinerização com Docker
10. O Dockerfile explicado linha por linha
11. O docker-compose.yml explicado linha por linha
12. Como executar o sistema
13. Resultado dos testes
14. Sobre o armazenamento das tarefas
15. Histórico de versões do repositório
16. Considerações finais

1. Como ler este documento

Este documento descreve o TaskTracker por inteiro, da concepção à entrega final, e foi escrito para que um profissional que nunca teve contato com o projeto consiga compreendê-lo e se situar diante da configuração desenvolvida, sem precisar de explicação adicional de quem o construiu.

A leitura está organizada em três blocos, que correspondem às três fases do projeto:

1 - As seções 3 a 6 tratam do planejamento lógico, ou seja, do raciocínio anterior a qualquer linha de código: qual problema o sistema resolve, quais dados ele manipula, quais regras ele obedece e como o repositório foi desenhado.

2 - As seções 7 e 8 tratam da implementação: o código-fonte módulo por módulo, com a função de cada arquivo, e o controle de versão adotado.

3 - As seções 9 a 13 tratam da conteinerização: por que ela foi feita, o conteúdo de cada arquivo criado, explicado linha por linha, e como executar o sistema.

Quem precisar apenas colocar o sistema para rodar pode ir direto à seção 12. Quem precisar entender uma decisão técnica específica encontrará a justificativa junto da linha correspondente, nas seções 10 e 11.

---

2. Visão geral do sistema

O TaskTracker é um sistema de gerenciamento de tarefas que funciona inteiramente em linha de comando. O usuário interage com ele por um menu de texto no terminal, escolhendo opções numeradas.

Em uma frase: o sistema recebe tarefas digitadas pelo usuário, valida cada informação, guarda as tarefas em memória durante a execução, exibe-as ordenadas por prioridade e informa quantas estão pendentes e quantas foram concluídas.

Características técnicas em resumo

1 - Linguagem: Python 3, desenvolvido e testado na versão 3.13.16 e executado em container na versão 3.11.

2 - Dependências externas: nenhuma. O sistema utiliza exclusivamente a biblioteca padrão do Python.

3 - Interface: terminal, por meio das funções nativas input e print.

4 - Armazenamento: em memória, em uma lista de dicionários, pelo tempo em que o programa estiver aberto.

5 - Organização: quatro módulos, separados por responsabilidade, dentro da pasta src.

6 - Execução: direta pelo Python ou isolada em container Docker.

As cinco funcionalidades

1 - Cadastrar nova tarefa. Solicita título, descrição, prioridade e data limite, valida cada entrada e registra a tarefa com a situação "Pendente".

2 - Visualizar tarefas cadastradas. Exibe todas as tarefas numeradas e ordenadas por prioridade, mostrando título, descrição, prioridade, data limite e situação.

3 - Concluir uma tarefa. Altera a situação da tarefa escolhida de "Pendente" para "Concluída".

4 - Ver resumo das tarefas. Exibe o total de tarefas cadastradas, quantas estão pendentes e quantas já foram concluídas.

5 - Sair da aplicação. Encerra o programa.

---

3. Fase 1 — O problema e a solução proposta

A primeira fase do projeto foi dedicada ao planejamento lógico. Nenhuma linha de código foi escrita nesta etapa: o objetivo era definir o problema, as regras e o algoritmo em português, de forma que a implementação posterior fosse apenas a tradução desse raciocínio para uma linguagem de programação.

O problema identificado

No dia a dia, uma pessoa costuma acumular várias tarefas ao mesmo tempo: compromissos da faculdade, trabalhos com prazo, obrigações de casa e demandas do trabalho. Sem um sistema para organizar tudo isso, as tarefas ficam espalhadas em lugares diferentes, e dessa dispersão surgem três consequências práticas:

1 - Esquecimento. A tarefa existe, mas se perde no meio das outras informações e só é lembrada depois que o prazo passou.

2 - Falta de prioridade. Sem uma ordem clara, a pessoa gasta tempo com o que é menos importante enquanto o urgente fica parado.

3 - Falta de visão do progresso. A pessoa não consegue enxergar quanto já concluiu e quanto ainda falta.

A solução proposta

O TaskTracker responde a cada um dos três pontos de forma direta. O esquecimento é tratado pela centralização: todas as tarefas ficam em um único lugar, com data limite registrada. A falta de prioridade é tratada pela classificação obrigatória em Alta, Média ou Baixa, com a listagem sempre ordenada nessa sequência. A falta de visão do progresso é tratada pelo resumo, que informa a qualquer momento o total de tarefas, quantas estão pendentes e quantas foram concluídas.

A escolha pelo terminal, e não por uma interface gráfica ou web, foi deliberada. O objetivo da disciplina é a construção da lógica, e o terminal permite que toda a atenção fique na regra de negócio, sem o custo adicional de construir telas.

---

4. Fase 1 — Decomposição dos dados e regras de negócio

Os dados de uma tarefa

Cada tarefa foi decomposta em cinco informações, que juntas formam um registro completo:

1 - Título. Identificação curta da tarefa. É o campo que aparece em destaque na listagem.

2 - Descrição. Detalhamento livre do que precisa ser feito.

3 - Prioridade. Classificação da urgência, restrita a três valores: Alta, Média ou Baixa.

4 - Data limite. Prazo final da tarefa, no formato DD/MM/AAAA.

5 - Situação. Estado atual da tarefa: Pendente ou Concluída. Diferente dos demais, este campo não é digitado pelo usuário, é definido pelo próprio sistema.

Na implementação, cada tarefa virou um dicionário com essas cinco chaves, e o conjunto de todas elas virou uma lista. A escolha da dupla dicionário e lista é justificada na seção 7.

As nove regras de negócio

As regras abaixo foram definidas nesta fase e permanecem implementadas sem nenhuma alteração até a entrega final. Elas são a parte do projeto que não mudou em nenhuma das três fases:

Regra 1 - O título é obrigatório. Entradas vazias ou compostas apenas por espaços em branco são recusadas.

Regra 2 - A prioridade aceita exclusivamente Alta, Média e Baixa, com ou sem acento, em qualquer combinação de maiúsculas e minúsculas.

Regra 3 - Toda tarefa nasce automaticamente com a situação Pendente.

Regra 4 - Só é possível concluir uma tarefa que exista na lista.

Regra 5 - A data limite deve seguir o formato DD/MM/AAAA. Datas inexistentes, como 31/02/2026, são recusadas.

Regra 6 - Uma tarefa já concluída não pode ser concluída novamente.

Regra 7 - A listagem sempre exibe as tarefas na ordem Alta, Média e Baixa.

Regra 8 - O programa só encerra quando o usuário escolhe a opção Sair.

Regra 9 - Os dados ficam armazenados em memória durante a execução, em uma lista de dicionários.

---

5. Fase 1 — O algoritmo do sistema

O algoritmo foi escrito em português antes de virar código. Ele é reproduzido abaixo na mesma sequência em que o sistema funciona.

Fluxo principal

1 - Criar uma lista vazia, que será o repositório das tarefas durante a execução.

2 - Exibir a mensagem de boas-vindas.

3 - Exibir o menu com as cinco opções.

4 - Ler a opção digitada pelo usuário e verificar se ela está entre 1 e 5. Se não estiver, avisar e perguntar novamente, sem encerrar o programa.

5 - Executar a funcionalidade correspondente à opção escolhida.

6 - Voltar ao passo 3, repetindo indefinidamente, até que a opção escolhida seja a 5.

7 - Na opção 5, exibir a mensagem de despedida e encerrar.

Fluxo do cadastro de uma tarefa

1 - Perguntar o título. Enquanto a resposta estiver em branco, avisar e perguntar de novo (Regra 1).

2 - Perguntar a descrição, sem restrição de conteúdo.

3 - Perguntar a prioridade. Enquanto a resposta não for Alta, Média ou Baixa, avisar e perguntar de novo (Regra 2).

4 - Perguntar a data limite. Enquanto a resposta não for uma data real no formato DD/MM/AAAA, avisar e perguntar de novo (Regra 5).

5 - Definir a situação como Pendente, sem perguntar nada ao usuário (Regra 3).

6 - Guardar as cinco informações como um registro único e acrescentá-lo à lista.

7 - Confirmar o cadastro na tela, citando o título da tarefa.

Fluxo da visualização

1 - Se a lista estiver vazia, informar que não há tarefas cadastradas e voltar ao menu.

2 - Caso contrário, ordenar as tarefas por prioridade, na sequência Alta, Média e Baixa (Regra 7).

3 - Exibir cada tarefa numerada, com os cinco campos.

4 - Ao final, informar o total de tarefas exibidas.

Fluxo da conclusão

1 - Se a lista estiver vazia, informar que não há tarefas para concluir e voltar ao menu.

2 - Exibir a listagem, para que o usuário saiba qual número corresponde a cada tarefa.

3 - Perguntar o número da tarefa. Enquanto o número não corresponder a uma tarefa existente, avisar e perguntar de novo (Regra 4).

4 - Se a tarefa escolhida já estiver concluída, avisar e voltar ao menu (Regra 6).

5 - Caso contrário, alterar a situação para Concluída e confirmar na tela.

Fluxo do resumo

1 - Se a lista estiver vazia, informar que não há tarefas cadastradas.

2 - Caso contrário, contar o total de tarefas.

3 - Contar quantas estão com a situação Concluída.

4 - Obter o número de pendentes pela diferença entre o total e as concluídas.

5 - Exibir os três números.

---

6. Fase 1 — Mapeamento arquitetural do repositório

Ainda na fase de planejamento foi desenhada a estrutura de pastas e arquivos do repositório, de modo que a implementação já nascesse organizada. A estrutura abaixo é a do estado atual do projeto, ao final da Fase 3:

```
tasktracker-projeto/
├── README.md                      # Documentação principal da aplicação
├── Dockerfile                     # Receita de construção da imagem
├── docker-compose.yml             # Orquestração da execução interativa
├── .gitignore                     # Regras para ignorar arquivos temporários do Python
├── requirements.txt               # Dependências do projeto
├── docs/
│   ├── planejamento_logico.pdf    # Planejamento lógico entregue na Fase 1
│   ├── mapeamento_arquitetural.pdf  # Mapeamento da estrutura do repositório
│   └── documentacao.pdf           # Documentação técnica completa do projeto
└── src/
    ├── main.py                    # Ponto de entrada e laço do menu
    ├── menu.py                    # Exibe o menu e lê a opção digitada
    ├── tarefas.py                 # As quatro funcionalidades do sistema
    └── validacoes.py              # Regras de validação das entradas
```

A lógica por trás dessa organização

A raiz do repositório guarda apenas o que diz respeito ao projeto como um todo: a documentação principal, as regras de versionamento e os arquivos de conteinerização. É nela que um profissional chega primeiro, e é por isso que o README e os arquivos do Docker ficam ali, visíveis sem precisar abrir nenhuma pasta.

A pasta docs reúne a documentação formal das três fases, em PDF, preservando o registro de cada etapa do projeto.

A pasta src isola o código-fonte. Essa separação permite que o Dockerfile copie apenas o que é necessário para a execução, deixando documentação e arquivos de apoio fora da imagem.

Todos os itens exigidos nos documentos de orientação das Fases 2 e 3 estão presentes: README.md, .gitignore, docs/planejamento_logico.pdf, src/main.py, Dockerfile e docker-compose.yml. Os demais arquivos são os que haviam sido planejados no mapeamento arquitetural da Fase 1 e foram mantidos, de modo que o repositório corresponda exatamente ao que foi documentado nas fases anteriores. Nenhum arquivo das entregas anteriores foi removido.

7. Fase 2 — Implementação em Python

A segunda fase traduziu o algoritmo em código. A decisão estrutural mais relevante desta etapa foi dividir o programa em quatro módulos, cada um com uma responsabilidade única, em vez de concentrar tudo em um único arquivo.

A divisão segue a própria decomposição feita na Fase 1: o que é entrada e saída na tela ficou em um módulo, o que é regra de validação ficou em outro, o que é funcionalidade do sistema ficou em um terceiro, e o controle geral ficou no ponto de entrada. O benefício prático é que uma alteração em uma regra de validação não exige abrir o arquivo que desenha o menu.

A ordem de importação entre os módulos foi planejada para não formar ciclos, o que travaria a execução: main.py importa menu e tarefas; tarefas.py importa menu e validacoes; menu.py e validacoes.py não importam nenhum outro.

Os trechos de código reproduzidos a seguir omitem os comentários de cabeçalho de cada arquivo, por questão de espaço. Os arquivos completos, com todos os comentários, estão no repositório.

7.1 main.py — ponto de entrada

Responsável por criar a lista de tarefas, manter o menu em repetição e encaminhar cada opção escolhida para a função correspondente. Implementa as Regras 8 e 9.

```python
import menu
import tarefas as modulo_tarefas


def main():
    tarefas = []

    menu.exibir_boas_vindas()

    while True:
        menu.exibir_menu()
        opcao = menu.ler_opcao_menu()

        if opcao == "1":
            modulo_tarefas.cadastrar_tarefa(tarefas)
        elif opcao == "2":
            modulo_tarefas.listar_tarefas(tarefas)
        elif opcao == "3":
            modulo_tarefas.concluir_tarefa(tarefas)
        elif opcao == "4":
            modulo_tarefas.exibir_resumo(tarefas)
        elif opcao == "5":
            menu.exibir_despedida()
            break

if __name__ == "__main__":
    main()
```

Três pontos merecem explicação. O primeiro é a lista vazia criada na primeira linha da função: ela é o repositório de dados do sistema, e é passada como argumento para todas as funcionalidades, que a modificam diretamente. O segundo é o laço while True, que mantém o menu em repetição e só é interrompido pelo break da opção 5, atendendo à Regra 8. O terceiro é a importação com apelido, import tarefas as modulo_tarefas, necessária porque existe uma variável local chamada tarefas dentro da função: sem o apelido, o nome do módulo e o nome da lista entrariam em conflito.

A última linha, a verificação if __name__ == "__main__", garante que a função principal só seja executada quando o arquivo for rodado diretamente, e não quando for importado por outro código.

7.2 menu.py — interface com o usuário

Concentra tudo o que aparece na tela e a leitura da opção escolhida. Não conhece as tarefas nem as regras: apenas exibe e lê.

```python
def exibir_cabecalho(texto):
    print()
    print()
    print("=" * 50)
    print(texto.center(50))
    print("=" * 50)

def exibir_boas_vindas():
    exibir_cabecalho("BEM-VINDO AO TASKTRACKER")
    print("\nSistema de gerenciamento de tarefas")

def exibir_menu():
    exibir_cabecalho("TASKTRACKER - MENU PRINCIPAL")
    print("1 - Cadastrar nova tarefa")
    print("2 - Visualizar tarefas cadastradas")
    print("3 - Concluir uma tarefa")
    print("4 - Ver resumo das tarefas")
    print("5 - Sair da aplicação")
    print("-" * 50)

def ler_opcao_menu():
    while True:
        opcao = input("Digite a opção desejada: ").strip()
        if opcao in ("1", "2", "3", "4", "5"):
            return opcao
        print("Opção inválida, tente novamente.")

def exibir_despedida():
    exibir_cabecalho("ENCERRANDO O TASKTRACKER")
    print("Até logo!")
    print()
```

A função exibir_cabecalho é usada por todas as telas do sistema e resolve um detalhe que deu trabalho durante o desenvolvimento. O método center centraliza o texto em uma largura de 50 caracteres, a mesma das linhas de igual que formam a moldura. As duas linhas em branco que criam o espaçamento entre blocos são impressas dentro da própria função, e não acrescentadas ao texto recebido: quando as quebras de linha faziam parte do texto, o método center as contava como caracteres e desalinhava o título.

A função ler_opcao_menu implementa a validação do menu. Ela só devolve o valor digitado quando ele está entre 1 e 5; em qualquer outro caso avisa e pergunta novamente, sem encerrar o programa.

7.3 tarefas.py — as funcionalidades

Reúne as quatro funcionalidades do sistema e as constantes que sustentam as regras de ordenação e de situação.

```
import menu
import validacoes

ORDEM_PRIORIDADE = {
    "Alta": 1,
    "Média": 2,
    "Baixa": 3,
}

SITUACAO_INICIAL = "Pendente"
SITUACAO_CONCLUIDA = "Concluída"

def cadastrar_tarefa(tarefas):
    menu.exibir_cabecalho("CADASTRO DE NOVA TAREFA")

    titulo = validacoes.ler_titulo()
    descricao = validacoes.ler_descricao()
    prioridade = validacoes.ler_prioridade()
    data_limite = validacoes.ler_data_limite()

    tarefa = {
        "titulo": titulo,
        "descricao": descricao,
        "prioridade": prioridade,
        "data_limite": data_limite,
        "situacao": SITUACAO_INICIAL,
    }

    tarefas.append(tarefa)
    print()
    print('Tarefa "{}" cadastrada com sucesso!'.format(titulo))

def ordenar_por_prioridade(tarefas):
    return sorted(tarefas, key=lambda t: ORDEM_PRIORIDADE[t["prioridade"]])
```

O dicionário ORDEM_PRIORIDADE é a tradução da Regra 7 para código. Ele atribui um peso numérico a cada prioridade, e a função ordenar_por_prioridade usa esse peso como critério de ordenação, devolvendo uma cópia ordenada sem alterar a lista original.

A função cadastrar_tarefa não valida nada por conta própria: ela delega cada campo ao módulo de validações e só monta o registro quando todas as respostas já estão corretas. A situação é preenchida pela constante SITUACAO_INICIAL, sem passar pelo usuário, o que implementa a Regra 3.

```
def listar_tarefas(tarefas):
    menu.exibir_cabecalho("SUAS TAREFAS")

    if len(tarefas) == 0:
        print("Nenhuma tarefa cadastrada até o momento.")
        return

    lista_ordenada = ordenar_por_prioridade(tarefas)

    for numero, tarefa in enumerate(lista_ordenada, start=1):
        print()
        print("[{}] {}".format(numero, tarefa["titulo"]))
        print("    Descrição...: {}".format(tarefa["descricao"]))
        print("    Prioridade..: {}".format(tarefa["prioridade"]))
        print("    Data limite.: {}".format(tarefa["data_limite"]))
        print("    Situação....: {}".format(tarefa["situacao"]))

    print()
    print("-" * 50)
    print("Total de tarefas exibidas: {}".format(len(lista_ordenada)))
```

A listagem trata primeiro o caso da lista vazia e encerra ali mesmo, com return, evitando exibir uma tela de resultados sem conteúdo. A função enumerate com start=1 numera as tarefas a partir de 1, e não de 0, porque esse número é o que o usuário vai digitar na hora de concluir uma tarefa.

```
def concluir_tarefa(tarefas):
    if len(tarefas) == 0:
        menu.exibir_cabecalho("CONCLUIR TAREFA")
        print("Não há tarefas para concluir.")
        return

    listar_tarefas(tarefas)
    lista_ordenada = ordenar_por_prioridade(tarefas)

    print()
    numero = validacoes.ler_numero_tarefa(len(lista_ordenada))
    tarefa = lista_ordenada[numero - 1]

    if tarefa["situacao"] == SITUACAO_CONCLUIDA:
        print()
        print("Esta tarefa já foi concluída anteriormente.")
        return

    tarefa["situacao"] = SITUACAO_CONCLUIDA
    print()
    print('Tarefa "{}" foi concluída!'.format(tarefa["titulo"]))
```

A conclusão exibe a listagem antes de perguntar o número, para que o usuário veja a numeração atualizada. O detalhe técnico relevante está na linha que acessa lista_ordenada[numero - 1]: como a lista ordenada contém referências aos mesmos dicionários da lista original, alterar a situação ali altera a tarefa de verdade, e não uma cópia. A verificação da situação antes da alteração implementa a Regra 6.

```
def exibir_resumo(tarefas):
    menu.exibir_cabecalho("RESUMO GERAL")

    if len(tarefas) == 0:
        print("Nenhuma tarefa cadastrada até o momento.")
        return

    total = len(tarefas)
    concluidas = 0
    for tarefa in tarefas:
        if tarefa["situacao"] == SITUACAO_CONCLUIDA:
            concluidas += 1
    pendentes = total - concluidas

    print("Total de tarefas...: {}".format(total))
    print("Pendentes.........: {}".format(pendentes))
    print("Concluídas.........: {}".format(concluidas))
```

O resumo percorre a lista contando as tarefas concluídas e obtém as pendentes pela diferença, em vez de percorrer a lista duas vezes.

7.4 validacoes.py — regras de validação

Concentra as regras que decidem se uma entrada digitada é aceitável. Cada função pergunta ao usuário e só devolve o valor quando ele é válido.

```
from datetime import datetime

PRIORIDADES_VALIDAS = {
    "alta": "Alta",
    "media": "Média",
    "média": "Média",
    "baixa": "Baixa",
}

def ler_titulo():
    while True:
        titulo = input("Título da tarefa: ").strip()
        if titulo == "":
            print("Erro: o título não pode ficar em branco. Tente novamente.")
            continue
        return titulo

def ler_descricao():
    return input("Descrição da tarefa: ").strip()

def ler_prioridade():
    while True:
        digitado = input("Prioridade (Alta / Média / Baixa): ").strip().lower()
        if digitado in PRIORIDADES_VALIDAS:
            return PRIORIDADES_VALIDAS[digitado]
        print("Erro: a prioridade deve ser Alta, Média ou Baixa. Tente novamente.")
```

O dicionário PRIORIDADES_VALIDAS resolve dois problemas de uma vez. Ele aceita o que o usuário digita em várias formas, com acento, sem acento, em maiúsculas ou minúsculas, e devolve sempre a forma padronizada. É isso que garante que o dicionário ORDEM_PRIORIDADE, no outro módulo, sempre encontre a chave esperada. O método strip remove espaços acidentais no início e no fim, e o lower converte para minúsculas antes da comparação.

```
def ler_data_limite():
    while True:
        data = input("Data limite (DD/MM/AAAA): ").strip()
        try:
            datetime.strptime(data, "%d/%m/%Y")
            return data
        except ValueError:
            print("Erro: use o formato DD/MM/AAAA. Tente novamente.")

def ler_numero_tarefa(total_de_tarefas):
    while True:
        digitado = input("Número da tarefa que deseja concluir: ").strip()

        if not digitado.isdigit():
            print("Erro: número de tarefa inválido. Tente novamente.")
            continue

        numero = int(digitado)
        if numero < 1 or numero > total_de_tarefas:
            print("Erro: número de tarefa inválido. Tente novamente.")
            continue

        return numero
```

A validação da data é o ponto mais interessante do módulo. A função strptime, da biblioteca padrão, tenta converter o texto digitado em uma data real segundo o formato informado. Se não conseguir, ela levanta um erro do tipo ValueError, que é capturado pelo bloco except. Esse mecanismo verifica duas coisas ao mesmo tempo: se o formato está correto e se a data existe de fato. Por isso uma entrada como 31/02/2026 é recusada, mesmo tendo o formato certo.

A validação do número da tarefa verifica primeiro se o texto digitado é composto apenas por dígitos, com isdigit, e só depois converte para número inteiro e compara com o intervalo válido. A ordem importa: tentar converter um texto como "abc" diretamente para número provocaria um erro e interromperia o programa.

7.5 Arquivos de apoio

O requirements.txt é o arquivo onde normalmente se listam as bibliotecas externas que o projeto precisa instalar. Neste caso ele contém apenas comentários, registrando que o projeto utiliza somente a biblioteca padrão do Python e informando a versão usada no desenvolvimento. A versão do Python foi registrada como comentário, e não como uma linha comum, porque uma linha comum seria interpretada como o nome de um pacote a instalar.

O .gitignore lista o que o Git deve ignorar. O caso principal é a pasta __pycache__, gerada automaticamente pelo Python para acelerar a execução seguinte. Esses arquivos são reconstruídos sozinhos e variam de máquina para máquina, de modo que versioná-los apenas poluiria o repositório.

8. Fase 2 — Versionamento com Git e GitHub

O projeto é versionado com Git e hospedado em repositório público no GitHub. A opção pelo repositório público foi deliberada: ela permite que qualquer pessoa acesse o código sem precisar ser cadastrada como colaboradora.

Antes do primeiro commit, foi verificada a instalação do Git e configurada a identidade que assina as alterações, com os comandos abaixo. Sem essa configuração, o Git recusa o commit, porque não sabe a quem atribuir a autoria:

```
git --version
git config --global user.name "EduardoHenrique"
git config --global user.email "eduardo.henriques@sempreceub.com"
```

A partir daí, o fluxo de trabalho seguiu sempre o mesmo ciclo: verificar o que mudou, selecionar os arquivos da alteração, registrar o commit com uma mensagem descritiva e enviar ao repositório remoto.

```
git status
git add <arquivos>
git commit -m "mensagem descritiva"
git log --oneline
git push
```

Os commits foram organizados por propósito, e não por momento de trabalho. Em vez de um único commit com tudo, cada conjunto coerente de alterações recebeu o seu, de modo que o histórico conte a construção do projeto de forma legível. O histórico completo está na seção 15.

---

9. Fase 3 — Conteinerização com Docker

Por que conteinerizar

Ao final da Fase 2 o sistema funcionava, mas com uma dependência silenciosa: a máquina de quem o executava. Era necessário ter o Python instalado, em uma versão compatível, e torcer para que nenhuma configuração local interferisse. Essa é a origem do problema clássico de que um programa funciona na máquina de quem o escreveu e falha na de quem o recebe.

A conteinerização elimina essa dependência. Em vez de pedir que a outra pessoa prepare o ambiente, o ambiente passa a ser descrito em arquivo e entregue junto com o código. Quem recebe o projeto executa dois comandos e obtém exatamente o mesmo ambiente, com a mesma versão do Python, independentemente do que tenha instalado.

Conceitos utilizados

Os termos abaixo são usados ao longo das próximas seções:

1 - Imagem. É um modelo pronto e congelado do ambiente da aplicação. Contém o sistema operacional reduzido, o Python na versão escolhida e o código-fonte do projeto. A imagem não executa nada sozinha, ela é apenas o molde.

2 - Container. É a imagem em execução. É possível criar vários containers a partir da mesma imagem, e cada um é isolado dos demais e da máquina onde está rodando.

3 - Dockerfile. É o arquivo de texto com as instruções de construção da imagem. Funciona como uma receita, lida de cima para baixo.

4 - Camada. Cada instrução do Dockerfile gera uma camada da imagem. O Docker guarda essas camadas e as reaproveita nas construções seguintes, o que torna o processo muito mais rápido a partir da segunda vez.

5 - Build. É o ato de construir a imagem a partir do Dockerfile.

6 - Orquestração. É a configuração da forma como o container deve subir, guardada em arquivo para não precisar ser digitada a cada execução.

7 - Volume. É uma pasta da máquina ligada ao container, usada quando os dados precisam sobreviver ao encerramento dele. Este projeto não utiliza volumes, pelo motivo explicado na seção 14.

Preparação do ambiente

A instalação do Docker Desktop foi recusada na primeira tentativa, com a mensagem de que o recurso do Windows chamado Virtual Machine Platform não estava habilitado. Esse recurso é o que permite ao Docker executar containers Linux dentro do Windows.

A solução foi habilitá-lo pelo PowerShell aberto como administrador, com o comando abaixo, seguido da reinicialização do computador:

```
wsl --install
```

Depois da reinicialização, a instalação foi concluída normalmente. O registro deste problema é intencional: ele é comum em máquinas Windows e a solução não é óbvia a partir da mensagem de erro exibida pelo instalador.

Vale o registro de um ponto operacional: o Docker Desktop precisa estar aberto e com o motor em execução para que qualquer comando funcione. Com ele fechado, os comandos falham com uma mensagem sobre não conseguir conectar ao daemon.

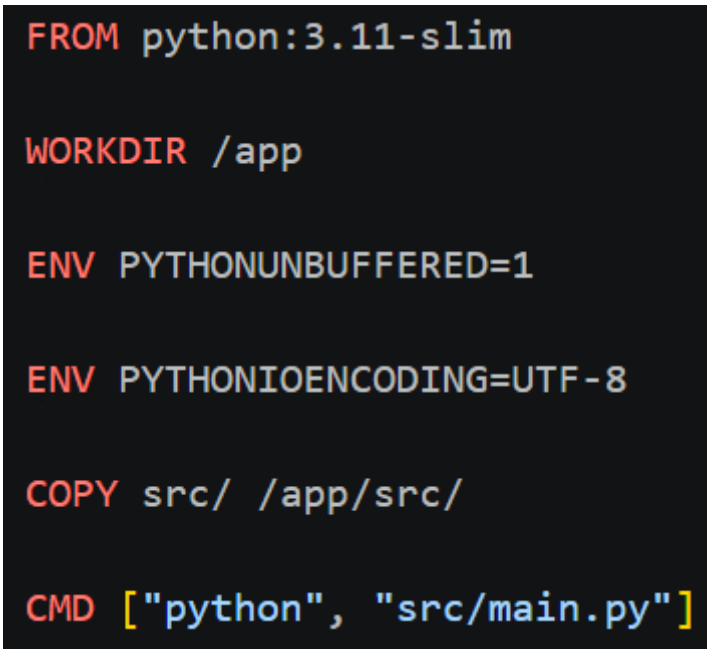
O que foi criado nesta fase

Foram criados dois arquivos na pasta raiz do repositório, ao lado do README.md. A localização na raiz é obrigatória, porque é ali que os comandos do Docker procuram por eles. Nenhum arquivo das fases anteriores foi alterado: o código-fonte dentro de src permaneceu exatamente como foi entregue na Fase 2, o que comprova que a conteinerização não exigiu mudanças na lógica da aplicação.

---

10. O Dockerfile explicado linha por linha

Este é o arquivo que o comando docker build utiliza. No repositório ele acompanha comentários explicativos em cada bloco. Abaixo estão apenas as instruções:



FROM python:3.11-slim

Define a imagem base, ou seja, o ponto de partida da construção. Em vez de montar um sistema do zero, o projeto parte de uma imagem oficial que já traz o Python 3.11 instalado.

A variação slim foi escolhida de forma deliberada. A imagem padrão do Python traz muitas ferramentas de sistema que o TaskTracker não utiliza, enquanto a slim traz o interpretador e apenas o essencial do sistema operacional. O resultado é uma imagem final consideravelmente menor e um build mais rápido. Uma terceira variação, a alpine, é ainda menor, mas foi descartada por usar uma biblioteca de sistema diferente, que costuma gerar incompatibilidades desnecessárias em um projeto deste porte.

WORKDIR /app

Define a pasta /app como diretório de trabalho dentro do container. A partir desta linha, qualquer comando executado parte dessa pasta, que funciona como a raiz do projeto dentro do ambiente isolado.

Essa instrução também cria a pasta caso ela ainda não exista. Sem ela, os arquivos seriam copiados para a raiz do sistema do container, misturados às pastas do próprio sistema operacional.

ENV PYTHONUNBUFFERED=1

Define uma variável de ambiente que desliga o buffer de saída do Python.

Por padrão, o Python acumula o que seria exibido na tela e só entrega esse conteúdo em blocos, para ganhar desempenho. Em um programa comum isso não faz diferença, mas o TaskTracker é interativo: ele imprime uma pergunta e para, esperando o usuário digitar. Com o buffer ligado, a pergunta pode ficar retida na memória e a tela aparecer vazia, dando a impressão de que o programa travou. Esta linha garante que cada mensagem apareça no instante exato em que é impressa.

ENV PYTHONIOENCODING=UTF-8

Define a codificação de caracteres usada na entrada e na saída do programa.

O TaskTracker exibe palavras acentuadas, como Média, Situação e Concluída. Como a imagem slim é um ambiente reduzido, esta linha garante de forma explícita que os acentos sejam exibidos corretamente dentro do container, independentemente da configuração de idioma da máquina onde ele estiver rodando.

COPY src/ /app/src/

Copia a pasta src da máquina local para dentro da imagem, no caminho /app/src.

A cópia é restrita ao código-fonte de propósito. A documentação, os PDFs da pasta docs e os arquivos de configuração do Git não têm utilidade dentro do container e ficam de fora, o que mantém a imagem leve. Como a pasta inteira é copiada, os quatro módulos do projeto chegam juntos, o que preserva as importações entre eles.

CMD ["python", "src/main.py"]

Define o comando executado automaticamente quando o container sobe. É o equivalente, dentro do ambiente isolado, ao comando que seria digitado no terminal da máquina local para iniciar o programa.

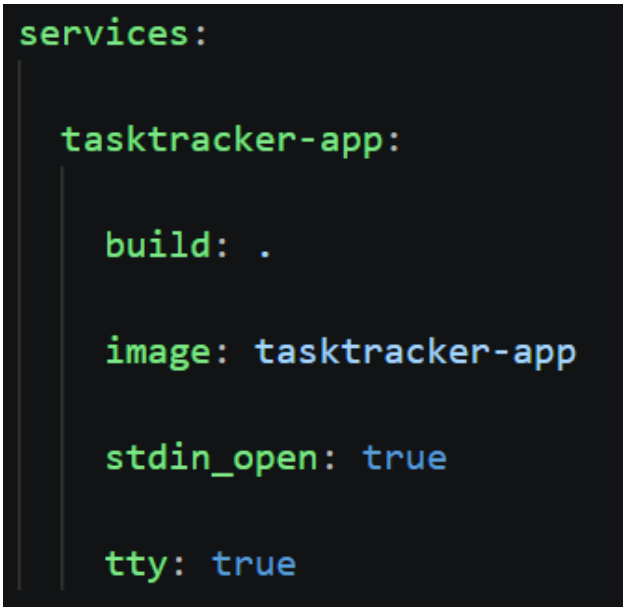
O caminho src/main.py é relativo ao diretório de trabalho definido anteriormente, resultando em /app/src/main.py.

Observação: não existe instrução de instalação de dependências neste arquivo porque o TaskTracker utiliza exclusivamente a biblioteca padrão do Python 3, conforme registrado no requirements.txt. Caso o projeto passasse a usar alguma biblioteca externa, seria necessário copiar o requirements.txt para a imagem e executar a instalação antes da cópia do código-fonte.

---

11. O docker-compose.yml explicado linha por linha

Este é o arquivo que o comando docker compose run utiliza. Se o Dockerfile é a receita da imagem, este é o maestro da execução: ele não repete nenhuma instrução de construção, apenas guarda a configuração de como o container deve subir.



Em arquivos com extensão .yml, o alinhamento das linhas é o que define a estrutura. Cada nível de indentação indica que a configuração pertence ao item acima dela, e o uso de tabulação no lugar de espaços torna o arquivo inválido.

services:

Abre a lista de serviços do projeto. Cada serviço corresponde a um container que o Docker Compose sabe subir. Este projeto tem um único serviço, porque é uma aplicação de terminal, sem banco de dados ou servidor acoplado.

tasktracker-app:

É o nome do serviço. É por este nome que o container é chamado na hora de executar, no comando docker compose run tasktracker-app.

build: .

Informa que a imagem deve ser construída a partir do Dockerfile encontrado nesta mesma pasta. O ponto representa o diretório atual. Essa linha é o que conecta os dois arquivos de conteinerização.

image: tasktracker-app

Define o nome da imagem gerada. Sem essa linha, o Docker Compose criaria um nome automático derivado do nome da pasta, mais difícil de identificar na listagem de imagens. Com ela, a imagem construída pelo Compose recebe exatamente o mesmo nome da imagem construída manualmente pelo docker build, o que mantém as duas formas de execução coerentes entre si.

stdin_open: true

Mantém o canal de entrada do container aberto. Corresponde ao parâmetro -i do comando docker run.

Esta é a configuração mais importante do arquivo. O TaskTracker conversa com o usuário por meio da função input, e um container sobe isolado por padrão, sem canal de entrada. Sem esta linha, o programa iniciaria, não conseguiria receber nada do teclado e encerraria sozinho.

tty: true

Reserva um terminal virtual para o container. Corresponde ao parâmetro -t do comando docker run. É o que faz o menu, as linhas de separação e as mensagens aparecerem formatados na tela, do mesmo jeito que aparecem quando o programa roda diretamente na máquina.

Observação: o arquivo não declara a chave version, presente em exemplos mais antigos de Docker Compose. Nas versões atuais essa chave é obsoleta e sua presença gera um aviso na execução.

---

12. Como executar o sistema

Pré-requisito: ter o Docker instalado e em execução. No Windows, o Docker Desktop precisa estar aberto, com a mensagem "Engine running" visível. Todos os comandos devem ser executados a partir da pasta raiz do projeto, a mesma onde estão o Dockerfile e o docker-compose.yml.

12.1 Obter o projeto

```
git clone https://github.com/EduardHenriqueCEUB/tasktracker-projeto.git
cd tasktracker-projeto
```

12.2 Construir a imagem

```
docker build -t tasktracker-app .
```

O parâmetro -t define o nome da imagem e o ponto final indica que o Dockerfile está no diretório atual. Na primeira execução o Docker baixa a imagem base do Python, o que leva alguns minutos. Nas execuções seguintes ele reaproveita as camadas já construídas e o processo é quase instantâneo.

12.3 Rodar o container em modo isolado

```
docker run -it --rm tasktracker-app
```

O parâmetro -i mantém o canal de entrada aberto e o -t reserva um terminal, o que permite digitar as opções do menu dentro do container. O --rm apaga o container automaticamente ao sair da aplicação, evitando o acúmulo de containers parados na máquina.

12.4 Rodar via orquestração (forma recomendada)

```
docker compose run --rm tasktracker-app
```

Esta é a forma mais simples de executar o sistema. O Docker Compose lê o arquivo docker-compose.yml, constrói a imagem caso ela ainda não exista e sobe o container já no modo interativo, sem que o usuário precise informar os parâmetros manualmente.

12.5 Execução sem Docker

O sistema continua podendo ser executado diretamente, para quem tiver o Python 3 instalado. Não é necessário instalar dependências nem criar ambiente virtual:

```
python src/main.py
```

12.6 Comandos de apoio

Comandos que não são necessários para executar o sistema, mas ajudam a acompanhar o que o Docker está fazendo:

```
docker images                              lista as imagens da maquina
docker ps -a                               lista os containers, inclusive encerrados
docker build --no-cache -t tasktracker-app . reconstroi ignorando o cache
docker rmi tasktracker-app                 remove a imagem do projeto
```

---

13. Resultado dos testes

As três formas de execução foram testadas e funcionaram sem erros. Ao subir o container, o sistema exibe o menu principal normalmente:

```
==================================================
          TASKTRACKER - MENU PRINCIPAL
==================================================
1 - Cadastrar nova tarefa
2 - Visualizar tarefas cadastradas
3 - Concluir uma tarefa
4 - Ver resumo das tarefas
5 - Sair da aplicação
--------------------------------------------------
Digite a opção desejada:
```

No teste de cadastro foi digitada de propósito uma prioridade inválida, para verificar se as validações criadas na Fase 2 continuavam funcionando dentro do ambiente isolado:

```
Título da tarefa: Estudar Docker
Descrição da tarefa: Revisar os comandos de build e run
Prioridade (Alta / Média / Baixa): urgente
Erro: a prioridade deve ser Alta, Média ou Baixa. Tente novamente.
Prioridade (Alta / Média / Baixa): alta
Data limite (DD/MM/AAAA): 30/11/2026

Tarefa "Estudar Docker" cadastrada com sucesso!
```

Dois comportamentos ficam comprovados nessa sequência. O primeiro é que a validação recusou a entrada inválida e repetiu a pergunta, sem encerrar o programa. O segundo é que a palavra "alta", digitada em minúsculas, foi aceita e padronizada para "Alta", conforme previsto na Regra 2.

Em seguida, a listagem exibiu a tarefa com todos os campos preenchidos e com a acentuação correta, confirmando o efeito da variável de codificação definida no Dockerfile:

```
==================================================
                SUAS TAREFAS
==================================================

[1] Estudar Docker
   Descrição...: Revisar os comandos de build e run
   Prioridade..: Alta
   Data limite.: 30/11/2026
   Situação....: Pendente

--------------------------------------------------
Total de tarefas exibidas: 1
```

A saída pela opção 5 encerrou o programa e o container foi removido automaticamente, conforme esperado pelo parâmetro --rm. O comportamento foi idêntico ao da execução direta pelo Python, o que comprova que a conteinerização preservou integralmente a lógica construída nas fases anteriores.

---

14. Sobre o armazenamento das tarefas

As tarefas cadastradas existem apenas enquanto o programa está aberto. Elas ficam guardadas na memória, em uma lista criada no início da execução. Ao escolher a opção Sair, essa lista deixa de existir e as tarefas são perdidas. Ao abrir o programa novamente, ele começa vazio.

Esse comportamento é intencional e corresponde à Regra 9 definida no planejamento lógico da Fase 1. O escopo das entregas é a implementação da lógica, das validações e da conteinerização, e não a persistência dos dados.

Dentro do container a situação é a mesma, e fica ainda mais evidente: o container é um ambiente isolado e descartável, criado no momento da execução e removido ao final dela pelo parâmetro --rm. Tudo o que foi cadastrado durante aquela sessão desaparece junto com ele.

Para que as tarefas fossem mantidas entre uma execução e outra, seria necessário gravá-las fora do programa, em um dos seguintes destinos:

1 - Um arquivo no próprio computador, como uma planilha em formato CSV ou um arquivo de texto estruturado. É a solução mais simples: ao cadastrar uma tarefa, o programa escreveria uma nova linha no arquivo; ao iniciar, leria o arquivo e carregaria as tarefas de volta para a lista. Funciona bem para um único usuário e permite abrir os dados diretamente no Excel.

2 - Um banco de dados, como SQLite ou PostgreSQL. É a solução adequada quando várias pessoas usam o sistema ao mesmo tempo, quando o volume de tarefas cresce ou quando é preciso buscar e filtrar informações com rapidez. O banco também garante que dois cadastros simultâneos não se sobrescrevam e que os dados não sejam corrompidos.

No caso do container, o Docker resolveria isso com o recurso chamado volume, que liga uma pasta da máquina ao ambiente isolado, permitindo que o arquivo ou o banco continuem existindo mesmo depois que o container é removido.

Em ambos os casos, cada tarefa deixaria de ser apenas um registro temporário e passaria a ser um dado histórico. Esse é o ponto que liga o projeto à minha área de Ciência de Dados.

---

15. Histórico de versões do repositório

O repositório preserva o histórico completo das entregas, com os commits organizados por propósito:

1 - Adiciona documentacao, gitignore e documentos da Fase 1. Estrutura inicial do repositório e documentos do planejamento lógico.

2 - Implementa estrutura principal: ponto de entrada e menu. Os módulos main.py e menu.py.

3 - Adiciona funcionalidades das tarefas e regras de validacao. Os módulos tarefas.py e validacoes.py.

4 - Adiciona Dockerfile e docker-compose.yml para conteinerizacao da aplicacao. Arquivos de conteinerização da Fase 3.

5 - Atualiza README com o guia de execucao via Docker. Documentação da Fase 3.

A separação em dois commits na última fase foi proposital, para que o histórico registrasse de forma distinta a mudança de infraestrutura e a mudança de documentação.

---

16. Considerações finais

A conteinerização encerra o ciclo do projeto de uma forma que torna visível o que cada fase acrescentou. Na primeira, o TaskTracker era um raciocínio organizado no papel: um problema delimitado, nove regras e um algoritmo em português. Na segunda, passou a ser um programa que funcionava, mas preso a um computador específico e à versão do Python instalada nele. Na terceira, passou a ser um sistema que pode ser entregue: o ambiente de execução deixou de ser uma suposição sobre a máquina de quem roda e passou a ser parte do próprio projeto, descrito em arquivo e versionado junto com o código.

Foi também a fase que mais evidenciou a diferença entre escrever código e entregar software. O código-fonte não precisou de uma única alteração para ser conteinerizado, o que mostra que a separação em módulos feita na Fase 2, seguindo o planejamento da Fase 1, estava bem resolvida. O trabalho desta etapa foi inteiramente sobre o ambiente ao redor do código.

Um profissional que receba este repositório hoje tem, em um único lugar, o raciocínio que originou o sistema, o código que o implementa, a descrição do ambiente que o executa e as instruções para colocá-lo no ar em dois comandos. Era esse o objetivo desta documentação.